

Databases I - CPIT240

LAB-3

SQL Data Definition Language

(Create Table with Constraints)

Tables Used in this Part

EMPLOYEES

Column Name	Data Type	Nullable	Default	Primary Key
EMPLOYEE_ID	NUMBER(6,0)	No	-	1
FIRST_NAME	VARCHAR2(20)	Yes	-	-
LAST_NAME	VARCHAR2(25)	No	-	-
EMAIL	VARCHAR2(25)	No	-	-
PHONE_NUMBER	VARCHAR2(20)	Yes	-	-
HIRE_DATE	DATE	No	-	-
JOB_ID	VARCHAR2(10)	No	-	-
SALARY	NUMBER(8,2)	Yes	-	-
COMMISSION_PCT	NUMBER(2,2)	Yes	-	-
MANAGER_ID	NUMBER(6,0)	Yes	-	-
DEPARTMENT_ID	NUMBER(4,0)	Yes	-	-
				1 - 11

DEPARTMENTS

Column Name	Data Type	Nullable	Default	Primary Key
DEPARTMENT_ID	NUMBER(4,0)	No	-	1
DEPARTMENT_NAME	VARCHAR2(30)	No	-	-
MANAGER_ID	NUMBER(6,0)	Yes	-	-
LOCATION_ID	NUMBER(4,0)	Yes	-	-
				1 - 4

JOB_GRADES

Column Name	Data Type	Nullable	Default	Primary Key
GRADE_LEVEL	VARCHAR2(3)	Yes	-	-
LOWEST_SAL	NUMBER	Yes	-	-
HIGHEST_SAL	NUMBER	Yes	-	-
				1 - 3

EMPLOYEES

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
100	Steven	King	SKING	515.123.4567	06/17/1987	AD PRES	24000	-	-	90
101	Neena	Kochhar	NKOCHHAR	515.123.4568	09/21/1989	AD VP	17000	-	100	90
102	Lex	De Haan	LDEHAAN	515.123.4569	01/13/1993	AD VP	17000	-	100	90
103	Alexander	Hunold	AHUNOLD	590.423.4567	01/03/1990	IT PROG	9000	-	102	60
104	Bruce	Ernst	BERNST	590.423.4568	05/21/1991	IT PROG	6000	-	103	60
107	Diana	Lorentz	DLORENTZ	590.423.5567	02/07/1999	IT PROG	4200	-	103	60
124	Kevin	Mourgos	KMOURGOS	650.123.5234	11/16/1999	ST MAN	5800	-	100	50
141	Trenna	Rajs	TRAJS	650.121.8009	10/17/1995	ST CLERK	3500	-	124	50
142	Curtis	Davies	CDAVIES	650.121.2994	01/29/1997	ST CLERK	3100	-	124	50
143	Randall	Matos	RMATOS	650.121.2874	03/15/1998	ST CLERK	2600	-	124	50
144	Peter	Vargas	PVARGAS	650.121.2004	07/09/1998	ST CLERK	2500	-	124	50
149	Eleni	Zlotkey	EZLOTKEY	011.44.1344.429018	01/29/2000	SA MAN	10500	.2	100	80
174	Ellen	Abel	EABEL	011.44.1644.429267	05/11/1996	SA REP	11000	.3	149	80
176	Jonathon	Taylor	JTAYLOR	011.44.1644.429265	03/24/1998	SA REP	8600	.2	149	80
178	Kimberely	Grant	KGRANT	011.44.1644.429263	05/24/1999	SA REP	7000	.15	149	-
										row(s) 1 - 15 of 20 >

DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
50	Shipping	124	1500
60	IT	103	1400
80	Sales	149	2500
90	Executive	100	1700
110	Accounting	205	1700
190	Contracting	-	1700
			row(s) 1 - 8 of 8

JOB_GRADES

GRADE_LEVEL	LOWEST_SAL	HIGHEST_SAL
A	1000	2999
B	3000	5999
C	6000	9999
D	10000	14999
E	15000	24999
F	25000	40000
		row(s) 1 - 6 of 6

Constraints

- A constraint is a rule to which data must conform.
- The Oracle server uses constraints to prevent invalid data entry into tables.
- A CONSTRAINT clause is an optional part of a CREATE TABLE statement or ALTER TABLE statement.

Constraints

You can use constraints to do the following:

- **Enforce rules** on the data in a table whenever a row is inserted, updated, or deleted from that table.
- **Prevent the deletion** of a table if there are dependencies from other tables (FK constraints)

Constraints

A CONSTRAINT can be one of the following:

- **A column-level constraint**

It refers to **a single column** in the table and do not specify a column name (except check constraints).

- **A table-level constraint**

It refers to **one or more** columns in the table.

Constraints

Constraint	Description
NOT NULL	Specifies that this column cannot hold NULL values .
PRIMARY KEY	Specifies the column that uniquely identifies a row in the table. The identified columns must be defined as NOT NULL .
FOREIGN KEY	Specifies that the values in the column must correspond to values in a referenced primary key.
UNIQUE	Specifies that values in the column must be unique .
CHECK	Specifies rules for values in the column.

Constraints

Column-level constraint

```
CREATE TABLE table_name (  
  column1 datatype [CONSTRAINT  
  constraint_name] constraint_type,  
  column2 datatype [CONSTRAINT  
  constraint_name] constraint_type,  
  ....  
);
```

Table-level constraint

```
CREATE TABLE table_name (  
  column1 datatype ,  
  column2 datatype ,  
  ....  
  [CONSTRAINT constraint_name]  
  constraint_type (column,...),  
);
```


NOT NULL constraint

- Ensures that the column contains **no null values**.
- Columns **without** the NOT NULL constraint **can contain null** values by default.
- NOT NULL constraints must be defined **at the column level**.

NOT NULL constraint

EMPLOYEE_ID	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	DEPARTMENT_ID
100	King	SKING	515.123.4567	17-JUN-87	AD_PRES	24000	90
101	Kochhar	NKOCHHAR	515.123.4568	21-SEP-89	AD_VP	17000	90
102	De Haan	LDEHAAN	515.123.4569	13-JAN-93	AD_VP	17000	90
103	Hunold	AHUNOLD	590.423.4567	03-JAN-90	IT_PROG	9000	60
104	Ernst	BERNST	590.423.4568	21-MAY-91	IT_PROG	6000	60
178	Grant	KGRANT	011.44.1644.429263	24-MAY-99	SA_REP	7000	
200	Whalen	JWHALEN	515.123.4444	17-SEP-87	AD_ASST	4400	10

...
20 rows selected.

↑
NOT NULL constraint
(No row can contain
a null value for
this column.)

↑
**NOT NULL
constraint**

↑
**Absence of NOT NULL
constraint**
(Any row can contain
a null value for this
column.)

UNIQUE CONSTRAINT

- A UNIQUE constraint requires that every value in a column or set of columns (key) be **unique**.
- **No duplicate** values in a specified column or set of columns.

UNIQUE CONSTRAINT

EMPLOYEES

EMPLOYEE_ID	LAST_NAME	EMAIL
100	King	SKING
101	Kochhar	NKOCHHAR
102	De Haan	LDEHAAN
103	Hunold	AHUNOLD
104	Ernst	BERNST

...



INSERT INTO

208	Smith	JSMITH
209	Smith	JSMITH

← Allowed

← Not allowed:
already exists

UNIQUE constraint

PRIMARY KEY constraint

- PRIMARY KEY constraint creates a primary key for the table.
- Only one primary key can be created for each table.
- The PRIMARY KEY constraint is a column or set of columns that uniquely identifies each row in a table.
- This constraint enforces uniqueness of the column or column combination and ensures that no column that is part of the primary key can contain a null value.
- The table-level primary key definition allows you to include two columns in the primary key definition

PRIMARY KEY constraint

DEPARTMENTS

PRIMARY KEY

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
50	Shipping	124	1500
60	IT	103	1400
80	Sales	149	2500

...

Not allowed
(null value)



INSERT INTO

	Public Accounting		1400
50	Finance	124	1500

Not allowed
(50 already exists)

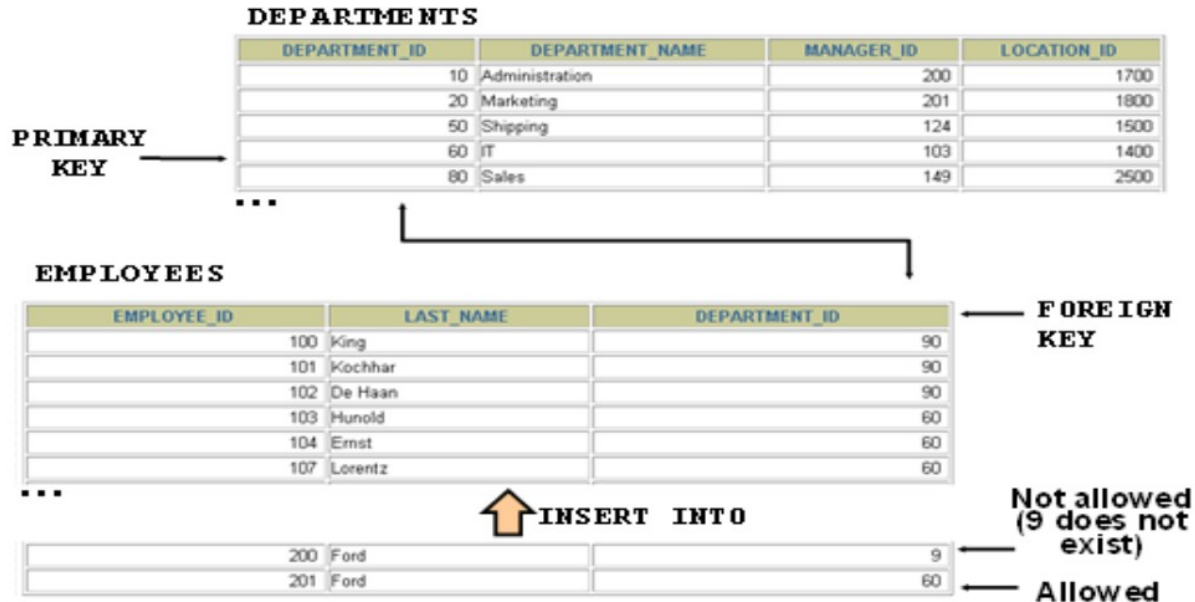
Check constraints

- The CHECK constraint defines a **search condition** (which is a boolean expression) condition that **each row must satisfy**.
- To satisfy the constraint, each row in the table must make the condition either **TRUE** or **unknown** (due to a null).
- The entire statement is **aborted** if any check constraint is violated.

FOREIGN KEY constraints

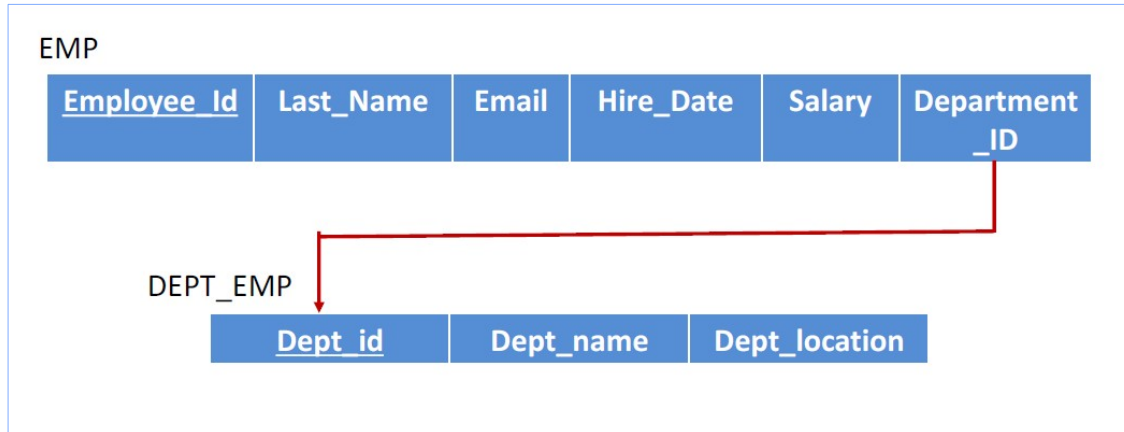
- FOREIGN KEY constraints can be defined **at the column or table constraint level**.
- **A composite foreign key** must be created by using the table-level definition.
- A foreign key value must **match** an existing value in the parent table or be NULL.

FOREIGN KEY constraints



Column Level Constraint

- We will use [the column level constraint](#) to implement the following schema:



TRY THIS

TRY THIS

1. Type the following to create the emp table with the NOT NULL constraint:

```
CREATE TABLE dept_emp(  
  dept_id NUMBER(4) PRIMARY KEY,  
  dept_name    VARCHAR2(25) UNIQUE,  
  location     VARCHAR2(25) NOT NULL);
```

2. Confirm that the table has been created,

```
DESCRIBE dept_emp
```

TRY THIS

TRY THIS

1. Type the following to create the emp table with the NOT NULL constraint:

```
CREATE TABLE emp(  
  employee_id  NUMBER(6) PRIMARY KEY,  
  last_name    VARCHAR2(25) NOT NULL,  
  email        VARCHAR2(25) UNIQUE,  
  hire_date    DATE DEFAULT sysdate,  
  salary       NUMBER(8,2) CHECK(salary>1000) ,  
  department_id NUMBER(4) references dept_emp(dept_id));
```

2. Confirm that the table has been created,
 DESCRIBE emp

Table-Level Constraint

- Table-level constraints are defined **at the end** of the table definition and **must refer** to the column or columns on which the constraint pertains in a set of parentheses.
- **NOTE:** Constraints that apply to more than one column must be defined at the table level.

TRY THIS

TRY THIS

1. Type the following to create a table-level constraint:

```
CREATE TABLE emp_table_level(  
  employee_id    NUMBER(6),  
  last_name      VARCHAR2(25),  
  email          VARCHAR2(25),  
  salary         NUMBER(8,2),  
  commission_pct NUMBER(2,2),  
  hire_date      DATE,  
  CONSTRAINT emp_email_unique UNIQUE(email));
```

2. Confirm that the table has been created:

```
DESCRIBE emp_table_level
```

ADDING A Constraint

- You can **add** a constraint **after creating a table**.
- Use the **ALTER TABLE** statement to:
 - **Add** or **drop** a constraint, but **not modify** its structure.
 - Add a **NOT NULL** constraint by using the **MODIFY** clause

ADDING A Constraint

Syntax

ALTER TABLE `tablename`

ADD [`CONSTRAINT constraint_name`] type (`column`);

ALTER TABLE `tablename`

MODIFY `column` NOT NULL

In the syntax:

- `tablename` Is the name of the table.
- `constraint_name` Is the name you want to give the constraint.
- `column` Is the name of the column.
- `type` Is the type of constraint, whether UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK .

TRY THIS

TRY THIS

1. Type the following to add a **CHECK** constraint:

```
ALTER TABLE emp_table_level  
ADD CONSTRAINT CHECK (salary > 0);
```

2. Type the following to add a **PRIMARY KEY** constraint:

```
ALTER TABLE emp_table_level  
ADD CONSTRAINT emp_empid_pk PRIMARY KEY (employee_id);
```

3. Type the following to add a **NOT NULL** constraint:

```
ALTER TABLE emp_table_level  
MODIFY hire_date NOT NULL;
```

4. Type the following to add a **NOT NULL** constraint:

```
ALTER TABLE emp_table_level  
MODIFY last_name NOT NULL;
```

5. Check that the table has been altered:

```
DESCRIBE emp_table_level;
```

Dropping A Constraint

- The `DROP CONSTRAINT` command is used to delete a `UNIQUE`, `PRIMARY KEY`, `FOREIGN KEY`, or `CHECK` constraint.

Dropping A Constraint

Syntax

- ALTER TABLE `tablename`
DROP PRIMARY KEY | UNIQUE (`column`) |
CONSTRAINT `constraint_name`;
- ALTER TABLE `tablename`
MODIFY `column` NULL

In the syntax:

- `tablename` Is the name of the table
- `column` Is the name of the column affected by the constraint
- `constraint_name` Is the name of the constraint

TRY THIS

TRY THIS

1. Type the following to drop the **emp_sal_min** constraint:

```
ALTER TABLE emp_table_level  
DROP CONSTRAINT emp_sal_min;
```

2. Type the following to drop a **PRIMARY KEY** constraint:

```
ALTER TABLE emp_table_level  
DROP PRIMARY KEY;
```

3. Type the following to drop a **NOT NULL** constraint:

```
ALTER TABLE emp_table_level  
MODIFY hire_date NULL;
```

4. Check that the table has been altered:

```
DESCRIBE emp_table_level;
```

Constraint Naming

It is a good practice to name the constraint due to the following:

- The constraint name will **appear** in the error message whenever a query violates it.
- It makes future modifications **easier**.

Examples

```
SQL> create table dept_emp(  
  2 dept_id number(6) constraint dept_emp_PK primary key,  
  3 dept_name varchar(25) unique,  
  4 location varchar(25));
```

Table created.

```
SQL> insert into dept_emp values(100,'IT','FCIT');
```

1 row created.

```
SQL> insert into dept_emp values(100,'IT','FCIT');  
insert into dept_emp values(100,'IT','FCIT')  
*
```

ERROR at line 1:

ORA-00001: unique constraint (USER1.DEPT_EMP_PK) violated

```
SQL> insert into dept_emp values(200,'IT','FCIT');  
insert into dept_emp values(200,'IT','FCIT')  
*
```

ERROR at line 1:

ORA-00001: unique constraint (USER1.SYS_C007230) violated